

Chapter 4. Model Serving: Architectures and Implementation

After we register the model, it will be ready for use by downstream systems. This raises the following question: how do we generate predictions and serve them to end users?

Many ML models are deployed as batch processes, where predictions are generated on a schedule (e.g., daily or hourly) and written to a database—either directly or transferred via mechanisms such as Secure File Transfer Protocol (SFTP). This database typically serves as the backend for an internal application, such as a demand forecasting or pricing system, that consumes the predictions for downstream decision making.

While implementing a full batch inference pipeline is one of the most common deployment patterns in production environments, we will not focus on it in this book. In practice, this setup would follow the same approach we demonstrated earlier when running the `predict` function in [Chapter 3](#). However, instead of running inference on observations from the test set, we would generate predictions on fresh, unseen, and unlabeled data, with the results persisted to a table for downstream consumption by other systems or applications.

In this chapter, I'll cover situations in which ML applications need to deliver predictions in real time.

There are three main scenarios we'll cover:

Model Serving

All the features are available to the model via the payload, and predictions are computed at the moment of request.

Serving precomputed batch predictions

Predictions are precomputed in advance and stored in a database, and they only get retrieved at the moment of request.

Predictions are computed at the moment of request. Some features are available via payload, and some features get retrieved at the moment of request. These features can be precomputed in a batch fashion, or they can be streaming features that are computed in real time and written to a feature store.

While I'll be covering all core deployment scenarios, I'll leave streaming features out of scope. In practice, true streaming features are rarely required for most AI applications. The majority of production systems work perfectly well with precomputed (batch) features that are refreshed on a schedule.

From an endpoint deployment perspective on Databricks, serving streaming features is almost identical to serving precomputed (batch) features. The model endpoint retrieves features in the same way, and the serving infrastructure does not fundamentally change based on how those features were produced.

The primary difference lies upstream: synchronization between the Databricks online feature store and the Delta table is continuous, rather than triggered by a batch job or scheduled process. This affects how feature freshness is maintained, but it does not change the deployment mechanics of the endpoint itself. We'll talk about these details in this chapter.

There's another important question in model deployment: who is responsible for it?

In some organizations, the same team that develops the model is also responsible for deploying and operating it in production. This can create tight feedback loops, reduce handoffs, and speed up iteration.

In other organizations, particularly larger enterprises and regulated industries, deployment is handled by dedicated ML engineering, platform, and infrastructure teams. Sometimes, this separation is mandated by risk management policies or compliance requirements. In these environments, having clearly defined responsibilities and controls is essential, and deployment ownership may intentionally sit outside the development team.

I recommend that you keep model development and production operations as close as you can. Tight feedback loops are what you are optimizing for, and the fewer handoffs there are between building a model and running it, the faster you will iterate and the easier debugging will become. Split ownership can still work, but it does not work for free. When responsibilities sit with multiple teams, you need strong tooling, shared visibility, and standardized processes to hold them together. Without these protocols, deployment slows down, communication overhead grows, and iteration cycles lengthen.

Model Serving is often challenging for data science teams, especially when they are expected to manage the deployment. This is where Databricks Model Serving can help.

With *Serverless Model and Feature Serving*, Databricks simplifies deployment infrastructure. You can create a model endpoint with just one Python command by using the Databricks SDK. This enables the following:

- Direct deployment by data science teams (when the organizational structure allows it)
- Standardization and governance of deployments by ML engineering teams at scale
- Clear ownership with less operational burden

Rather than prescribing a single “ideal” structure, the goal is to enable whichever team owns production deployment—data scientists, ML engineers, or a centralized platform team—to move quickly without compromising governance or reliability.

Let’s begin by covering how to deploy an endpoint in the simplest scenario.

Model Serving

In the first scenario, we’ll explore *Model Serving*, assuming that all required features are included in the request payload. [Figure 4-1](#) shows a typical automated pipeline for retraining and deploying a model in Databricks. This example references two components: *Lakeflow jobs*, which handle orchestration, and *Unity Catalog* (UC), which manages gov-

ernance and access. We first introduced these in [Chapter 1](#), and we will examine them in detail in [Chapter 5](#).

For Model Serving, we can perform both *model retraining* and *endpoint updates* within the same pipeline. A scheduled Lakeflow job runs data preprocessing and model retraining steps, and after that, the new model is registered in Unity Catalog. The pipeline then programmatically updates the real-time serving endpoint so applications always use the latest version of the model.

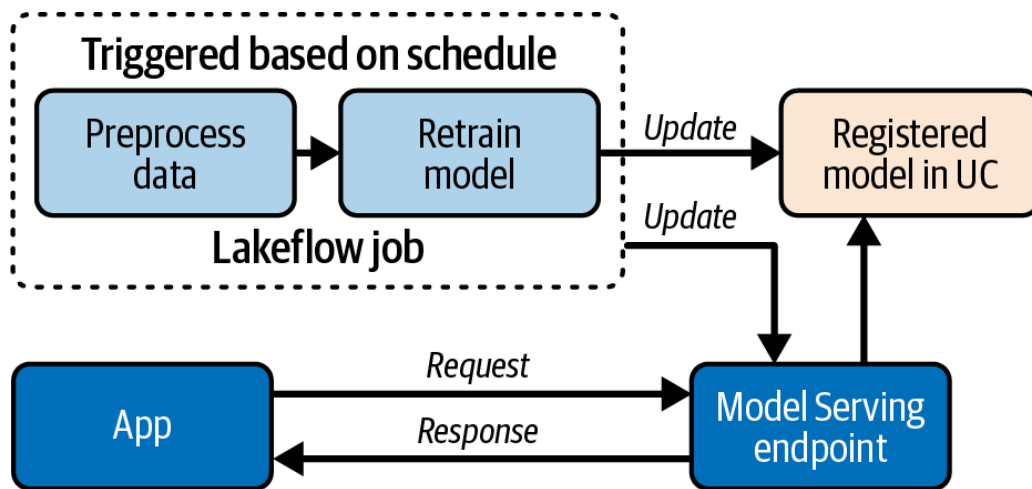


Figure 4-1. Simplified architecture for Model Serving

We will discuss this pipeline in depth in [Chapter 5](#), but for now, let's focus on the endpoint deployment itself. Our goal is to predict the nightly price of a hotel reservation using information the user provides, like the number of children and adults, the number of weekday and weekend nights, parking spaces available, special requests, and so on.

To deploy an endpoint for this use case, we first need a registered model. So let's grab the latest version of our wrapped model:

```
import mlflow
from mlflow import MlflowClient

from hotel_booking.config import ProjectConfig
from hotel_booking.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()

cfg = ProjectConfig.from_yaml("../project_config.yml")

catalog = cfg.catalog
```

```

schema = cfg.schema

model_name = "hotel_booking_pyfunc"

client = MlflowClient()
model_version = client.get_model_version_by_alias(
    alias="latest-model",
    name=f"{catalog}.{schema}.{model_name}")

```

Then, we'll need to define a list that consists of instances of `ServedEntityInput`. If we want to serve a model, we'll need to provide the following:

entity_name

This is the full name of the model in the following format:

`{catalog_name}.{schema_name}.{model_name}`.

entity_version

This is the specific version of the model to serve.

environment_vars

This is optional, but it's useful, for example, if we want to authenticate to another service and refer to Databricks secrets. We will use this feature in [Chapter 9](#).

workload_type

The default is a CPU, but various graphics processing unit (GPU) workload sizes are also available.

scale_to_zero_enabled

This controls whether the compute resources should scale down to zero when not in use. I recommend that you set this parameter to `True` for development and `False` for production, due to a cold-start problem.

Concurrency settings

You must define autoscaling ranges by specifying either a `workload_size` (e.g., Small, Medium, Large) or by setting `min_provisioned_concurrency` and `max_provisioned_concurrency`.

In the following example, we'll use some of these parameters:

```
from databricks.sdk.service.serving import ServedEntityInput

served_entities = [
    ServedEntityInput(
        entity_name=f"{catalog}.{schema}.{model_name}",
        scale_to_zero_enabled=True,
        workload_size="Small",
        entity_version=model_version.version,
    )
]
```

Now, let's talk about autoscaling:

- A single unit of provisioned concurrency can process one request at a time.
- The workload size corresponds to a range of provisioned concurrency units. The valid values are: Small (4 units), Medium (8–16 units), and Large (16–64 units). We can also use additional custom workload sizes when they're available in the workspace. They can be requested on demand.
- Autoscaling is based on the number of required units, which is calculated as *queries per second* $\times$ *model processing time*.

Let's look at an example. Imagine your service needs to handle 1,000 queries per second, and the model takes 0.02 seconds to process each query. To handle this load, the serving endpoint needs to be able to process 20 queries simultaneously. So in this case, you would provision 20 units of concurrency.

Endpoint Deployment

Let's deploy an endpoint using Databricks SDK. When creating an endpoint, we can customize it with a few optional settings:

Cost monitoring

We can add tags or a usage policy ID to monitor the costs associated with the endpoint.

Route optimization

When this is enabled, a unique, optimized URL is created for the endpoint, which can lower latency. A key detail to remember is that this feature only supports OAuth authentication, so you cannot use a personal access token (one that is generated via the workspace client) for testing. While we won't enable this in our example, it's often a good idea to enable it for production use.

Databricks Unity AI Gateway configuration

AI Gateway allows you to enable the following:

- Fallback to another served entity if a served entity fails with certain error codes
- AI guardrails for foundation model APIs, to prevent unsafe or unwanted data in requests and responses (which we'll cover in [Chapter 9](#))
- An inference table for logging all requests and responses for monitoring and audit purposes
- Rate limits that you can set to limit endpoint traffic
- Usage tracking in system tables, to monitor operational usage on endpoints

Here, we'll enable usage tracking and the inference table for our endpoint to use it for monitoring:

```
from databricks.sdk.service.serving import (
    AiGatewayConfig,
    AiGatewayInferenceTableConfig,
    AiGatewayUsageTrackingConfig,
)
ai_gateway_cfg = AiGatewayConfig(
    inference_table_config=AiGatewayInferenceTableConfig(
        enabled=True,
        catalog_name=catalog,
        schema_name=schema,
        table_name_prefix="hotel_booking_monitoring",
    ),
    usage_tracking_config=AiGatewayUsageTrackingConfig(enabled=True)
)
```

Next, we either create a new endpoint or update an existing one. In the configuration, we can optionally define traffic routing using `TrafficConfig`. In this example, we route 100% of the traffic to a single model. Providing a traffic configuration is optional; however, we include it here for clarity and demonstration purposes. In practice, you could serve multiple entities and define multiple traffic routes, for example, by splitting traffic 50/50 between two model versions. We'll return to traffic-splitting considerations when we cover A/B testing in the next section.

Finally, we specify the usage policy ID defined in the *project_config.yml* file, which allows us to track and attribute project-related costs:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import (
    EndpointCoreConfigInput,
    Route,
    TrafficConfig,
)

w = WorkspaceClient()
endpoint_name = "hotel-booking-pyfunc"
endpoint_exists = any(
    item.name == endpoint_name for item in w.serving_endpoints.list()
)

traffic_config=TrafficConfig(
    routes=[
        Route(
            served_model_name=f"{model_name}-{model_version.version}",
            traffic_percentage=100)
    ]
)

if not endpoint_exists:
    w.serving_endpoints.create(
        name=endpoint_name,
        config=EndpointCoreConfigInput(
            name=endpoint_name,
            served_entities=served_entities,
            traffic_config=traffic_config),
        ai_gateway=ai_gateway_cfg,
        budget_policy_id=cfg.usage_policy_id,
    )
else:
    w.serving_endpoints.update_config(
```

```

        name=endpoint_name, served_entities=served_entities
    )

```

Make sure the endpoint is ready before sending requests. You can check its status by using

```
workspace.serving_endpoints.get(name=endpoint_name).
```

Alternatively, instead of the `create()` and `update_config()` methods, you can use `create_and_wait()` and `update_config_and_wait()`, which block until the operation completes and the endpoint is ready.

Once the endpoint is deployed and ready, you can start sending inference requests. Databricks Model Serving uses MLflow Model Serving under the hood, and it therefore follows the same request payload format. For models that expect a Pandas DataFrame as input rather than Tensors, you must send requests by using either the `dataframe_split` or the `dataframe_records` format, with `dataframe_split` generally being the recommended option because it guarantees that the column ordering is preserved (see [Table 4-1](#)).

Table 4-1. Payload structure for Model Serving (from the [MLflow documentation](#))

Field	Description	Example
<code>dataframe_split</code>	A Pandas DataFrame in the split orientation	<pre>{"dataframe_split": pandas_df.to_dict(orient="split")}</pre>
<code>dataframe_records</code>	A Pandas DataFrame in the records orientation	<pre>{"dataframe_records": pandas_df.to_dict(orient="records")}</pre>
<code>instances</code>	Tensor input formatted according to TensorFlow Serving API	<pre>{"instances": [[1.0, 2.0, 5.0]]}</pre>
<code>inputs</code>	Same as instances but with a different key	<pre>{"inputs": [["Cheese"], ["and", "Crackers"]]}</pre>

Before we can send any requests, we need to authenticate. In the *notebooks/chapter4_1.model_serving.py* file, we can retrieve a personal access token and the host by using the workspace client:

```
host = w.config.host
token = w.tokens.create(lifetime_seconds=1200).token_value
```

You should not use personal access tokens for a production setting; instead, you must create a service principal and use its OAuth token. We'll discuss setting it up at the end of this chapter.

When calling the endpoint, we can provide the payload in two formats:

`dataframe_records` and `dataframe_split`:

```
payload = {
    "dataframe_records": [{
        'number_of_adults': 2,
        'number_of_children': 0,
        'number_of_weekend_nights': 0,
        'number_of_week_nights': 1,
        'car_parking_space': 0,
        'special_requests': 0,
        'lead_time': 103,
        'type_of_meal': 'Not Selected',
        'room_type': 'Room_Type 1',
        'arrival_month': 8,
        'market_segment_type': 'Online'
    }]
}

payload = {
    "dataframe_split": {
        'columns': ['number_of_adults', 'number_of_children',
                    'number_of_weekend_nights', 'number_of_week_nights',
                    'car_parking_space', 'special_requests', 'lead_time',
                    'type_of_meal', 'room_type', 'arrival_month',
                    'market_segment_type'],
        'data': [[2, 0,
                    0, 1,
                    0, 0, 103,
                    'Not Selected', 'Room_Type 1', 8,
                    'Online']]
    }
}
```

Then, we can use the `requests` library to call the endpoint:

```
import requests

serving_endpoint = f"{host}/serving-endpoints/hotel-booking-pyfunc/invocation

response = requests.post(
    serving_endpoint, headers={"Authorization": f"Bearer {token}"}, json=payload
)
```

The output contains the “predictions” key, and the content is defined by the PyFunc wrapper:

```
{"predictions": {"Total price per night": [104.98]}}
```

Deploying a model without proper validation can lead to costly delays. Spinning up an endpoint can take 10 minutes or more, and it’s an expensive interaction cycle. Luckily, Databricks provides mechanisms for debugging the endpoint before deployment.

Databricks also supports [express deployments](#), which significantly speed up the deployment process. It’s important to note that this feature is available only within the *serverless environment*. You must log and register the model from a serverless compute, which may not be suitable for certain ML workloads, as we discussed earlier. This is, however, a great feature for AI application deployment, and we’ll use it in the LLMOps chapters.

A/B Testing

In the section [“Endpoint Deployment”](#), we configured an endpoint with a `TrafficConfig` and routed all of its traffic to a single model. We noted there that you could instead split traffic across two models, say 50/50 between two versions, and that this is the simplest way to run an A/B test. It is not the only way, though, and it is not always the correct one. Before you rely on it, you should learn exactly what the split does and what it quietly fails to do.

The defining property of a traffic split is that model selection is determined independently for each request. As a result, the same customer may be routed to different model versions over the course of an experiment. This independence is what makes traffic splits easy to implement,

but it is also what makes them unsuitable for certain types of experiments.

The alternative is to derive the assignment from a stable identifier for the entity you care about, such as a customer ID (in our example, the booking ID). You hash the identifier and map the hash to a bucket, so the same identifier is always assigned to the same model for the duration of the experiment. Unlike a traffic split, the assignment is deterministic: the same booking, scored twice, always gets the same model.

Choosing a routing strategy

The rule for this is simple: randomize at the same level at which you will analyze the experiment.

The experimentation literature distinguishes between the randomization unit and the analysis unit.¹ The *randomization unit* is the entity assigned to a model. The *analysis unit* is the entity on which the outcome is measured. In a well-designed experiment, the two usually match.

Let's discuss what randomization unit to choose in different scenarios.

Randomization by customer

Most experiments do this because most business outcomes are measured by customer. Conversion, retention, repeat bookings, and revenue are all customer-level metrics. If a customer sees multiple models during the experiment, their behavior will no longer be attributable to a single group, and that will make the results harder to interpret. Customer-level assignment avoids this problem and keeps the customer experience consistent throughout the experiment.

Session-level assignment

This is the middle-of-the-road approach. Use it when the outcome is defined within a visit, rather than across visits. For example, if you care about session conversion or behavior during a single browsing session, you should assign each session to one model, but you can reassign a returning customer in a later session. This is also a practical choice when no stable customer identifier exists.

Request-level assignment

This is appropriate only when requests are genuinely independent. This is typically the case for backend model evaluations in which the prediction is invisible to the customer and does not influence their future behavior. Examples include latency measurements, throughput comparisons, offline batch scoring, and accuracy evaluation of independent transactions, documents, or events. Here, a simple traffic split is sufficient because the request itself is both the randomization unit and the analysis unit.

A useful test involves asking what entity appears in your final metric. If the metric is measured per request, you should split by request. If it is measured per session, split by session. If it is measured per customer, split by customer. The assignment mechanism should follow the unit of analysis, not the implementation convenience of a traffic router.

Moving assignment into the model logic

So far, we have shown you how you could configure traffic splitting programmatically at the endpoint level. That mechanism randomizes at the request level, which is fine when the request is both the unit of assignment and the unit of analysis

Now, consider an experiment in which the booking is the experimental unit. Every prediction associated with the same booking must be handled by the same model throughout the experiment. To assign consistently at the booking level, the assignment decision must move out of the endpoint configuration and into the model logic itself.

We will use the model we trained in the previous chapter as variant A:

```
from hotel_booking.config import ProjectConfig

cfg = ProjectConfig.from_yaml(
    config_path="../project_config.yml")

model_name_a = f"{cfg.catalog}.{cfg.schema}.hotel_booking_basic"
model_a_uri = f"models:{model_name_a}@latest-model"
```

Next, we train a second variant with a different set of hyperparameters. In practice, the difference between variants can be much larger: a different algorithm, a different feature set, or an entirely different preprocessing pipeline. But different hyperparameters are enough to illustrate the mechanism:

```
from pyspark.sql import SparkSession

from hotel_booking.data.data_loader import DataLoader
from hotel_booking.models.lightgbm_model import LightGBMModel

spark = SparkSession.builder.getOrCreate()
data_loader = DataLoader(spark=spark, config=cfg)
X_train, y_train, X_test, y_test = data_loader.split()

model_b = LightGBMModel(config=cfg)

model_b.train(
    X_train=X_train,
    y_train=y_train,
    parameters={"learning_rate": 0.01,
                "n_estimators": 1000,
                "max_depth": 6},
)
```

Then, we log and register variant B:

```
from hotel_booking.config import Tags
from hotel_booking.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()
tags = Tags(**{"git_sha": "1234567890abcd", "branch": "main"})

model_b.log_model(
    experiment_name="/Shared/hotel-booking-ab-testing",
    tags=tags,
    X_test=X_test,
    y_test=y_test,
    train_set_spark=data_loader.train_set_spark,
    train_query=data_loader.train_query,
    test_set_spark=data_loader.test_set_spark,
    test_query=data_loader.test_query,
)

model_name_b = f"{cfg.catalog}.{cfg.schema}.hotel_booking_ab_b"
```

```

model_b.register_model(model_name=model_name_b, tags=tags)
model_b_uri = f"models://{model_name_b}@latest-model"

```

Now, we create a wrapper model that contains the assignment logic. The wrapper receives the booking identifier as a parameter in the request payload, hashes it using `hashlib`, converts the hex digest to an integer, and assigns the request to variant A or B based on whether the result is even or odd. The hex digest is the MD5 hash written as a 32-character base-16 string, and `int(hash_id, 16)` reads that string back into a single large integer, where the 16 tells Python that the digits are hexadecimal so the characters a to f, which are worth 10 to 15, are valid.

Hashing is what makes the split well behaved. Raw booking identifiers often follow patterns, such as sequential numbering, whereas an MD5 hash maps them to values spread evenly across its output range. This means the odd/even outcome lands close to 50/50 and stays uncorrelated with any property of the booking. This is also deterministic, so the same booking identifier always maps to the same variant, regardless of how many requests it generates:

```

import hashlib

import mlflow
import numpy as np
import pandas as pd
from mlflow.pyfunc import PythonModelContext

class HotelBookingABTestWrapper(mlflow.pyfunc.PythonModel):
    def load_context(self, context: PythonModelContext) -> None:
        self.model_a = mlflow.sklearn.load_model(
            context.artifacts["model-a"])
        self.model_b = mlflow.sklearn.load_model(
            context.artifacts["model-b"])

    def predict(
        self,
        context: PythonModelContext,
        model_input: pd.DataFrame | np.ndarray,
        params: dict | None = None,
    ) -> dict:
        booking_id = str((params or {}).get("booking_id", ""))
        hashed_id = hashlib.md5(booking_id.encode("UTF-8")).hexdigest()

```

```

# Hex digest to int; even/odd gives a stable 50/50 split.
if int(hashlib.md5(model_input.encode()).hexdigest(), 16) % 2:
    predictions = self.model_a.predict(model_input)
    variant = "A"
else:
    predictions = self.model_b.predict(model_input)
    variant = "B"
prices = [round(float(pred) * 1.05, 2) for pred in predictions]
return {
    "Total price per night": prices,
    "model": variant,
    "booking_id": booking_id,
}

```

[Table 4-2](#) lists some examples of the assignment.

Table 4-2. Conversion of booking ID to hex digest and to model variant assignment

Booking ID	MD5 hex digest	int % 2	Variant
INN12345	5c8f093e729b3cccbcd92a48848c3fbd	1	A
INN12346	47ee65b0ba45f45b68a093dcbfd5f6b3	1	A
INN12347	b52126aac65831a322a21c43d1e8d58a	0	B
INN12348	1047b9493a5f98b4cd8a1e7d27059aea	0	B

We log, register, and deploy the wrapper model in the same way as the custom model from the previous section. The only difference at inference time is that the request payload must include the booking identifier so the wrapper can perform the assignment. The full implementation is available in the *chapter4_1a.ab_testing_demo.py* notebook.

In practice, organizations rarely implement model assignment this way. Instead, they integrate with an experimentation platform, either built in house or provided by a third-party service, that handles assignment through an API call and supports experiment tracking and statistical analysis.

Now that we've covered deployment and A/B testing, our next practical concern is debugging. Model Serving endpoints can fail for reasons that are not always obvious, and having a systematic approach to diagnosis will save us considerable time.

Debugging

A common check involves loading the model using

`mlflow.pyfunc.load_model()` and running test data through it. While this works in Visual Studio Code (VS Code) or a Databricks notebook, it doesn't fully simulate the serving environment and may miss issues related to packaging or dependencies.

A more reliable approach is to use `mlflow.models.predict()`:

```
import pandas as pd

input_example = pd.DataFrame(
    data=payload["dataframe_split"]["data"],
    columns=payload["dataframe_split"]["columns"]
)

model_uri = f"models://{catalog}.{schema}.{model_name}@latest-model"
mlflow.models.predict(model_uri, input_example)
```

Unlike `load_model()`, it builds a lightweight virtual environment based on the `conda_env` defined in your model's configuration. This makes it a much closer proxy to the actual serving container. Although `predict()` runs 5 to 10 times slower than `load_model()`, it's worth it. Catching environment issues early can save you hours of troubleshooting failed end-point deployments.

MLflow also includes handy functions to convert an input example to a serving input and to validate that input:

```
from mlflow.models import validate_serving_input
from mlflow.models import convert_input_example_to_serving_input

serving_payload = convert_input_example_to_serving_input(input_example)
validate_serving_input(model_uri, serving_payload)
```

Now that we have walked through the Model Serving implementation, let's check another common scenario: serving batch features.

Feature Serving

Serving batch predictions is probably one of the most popular and underestimated types of ML model deployment. Predictions are computed in advance with a batch process, stored in a transactional database, and retrieved upon request.

This architecture is very popular in the case of personal recommendations with low latency requirements. For example, an e-commerce store may recommend products to customers on various pages of its website.

Databricks Feature Serving is a great feature for use in this scenario (see [Figure 4-2](#)). A scheduled Lakeflow job preprocesses data, retrains the model, and generates predictions, which are stored in a feature table registered in the Unity Catalog. These features are then synced to an online store and served via a Feature Serving endpoint that's defined by a `FeatureSpec`. The application can query this endpoint in real time to retrieve the most up-to-date features for inference.

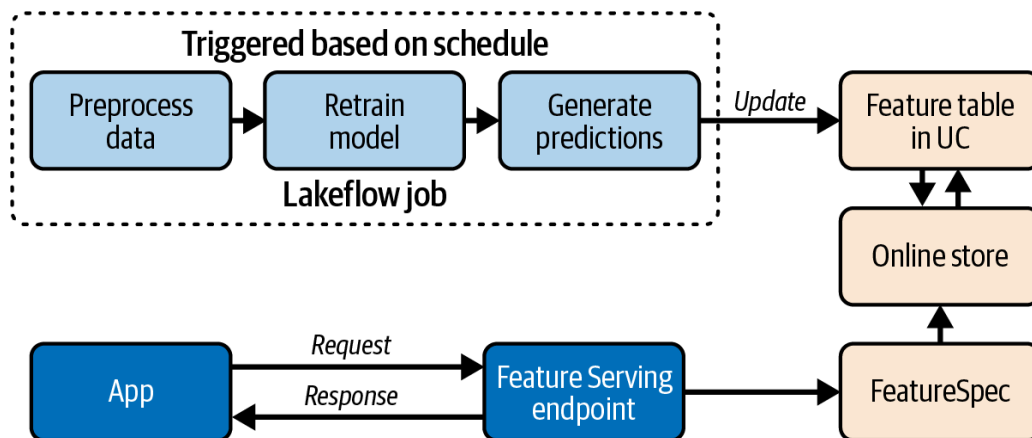


Figure 4-2. Simplified architecture for Feature Serving

When you're using Feature Serving, the payload doesn't need to include every feature the model requires. Values or predictions are precomputed in advance and stored in the online store, and you can retrieve them on demand by using just the `Booking_ID`.

Now, let's go through the `notebooks/chapter4_2.feature_serving.py` notebook, where we cover all the steps required for Feature Serving, starting

with the generating predictions step. We'll be using the `hotel_booking_basic` model, which we trained and registered earlier. We can easily access the latest registered version of this model by using the `@latest-model` alias.

NOTE

This notebook must be executed in a Databricks environment as it uses the `databricks-feature-engineering` package, which does not work from VS Code.

Creating a Feature Table

Let's go through the steps required to create a feature table. First of all, we load the input data into a `pyspark DataFrame` and only select the features required to generate model predictions. Then, we use `mlflow.pyfunc.spark_udf` to generate predictions at scale. The final output is a new `DataFrame` that contains just the `Booking_ID` and the predicted booking price, which is labeled `Predicted_BookingPrice`:

```
import mlflow
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, struct

from hotel_booking.config import ProjectConfig

cfg = ProjectConfig.from_yaml("../project_config.yml")
spark = SparkSession.builder.getOrCreate()

model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_basic"
model_uri = f"models:{model_name}@latest-model"
columns = cfg.cat_features + cfg.num_features

input_df = spark.table(f"{cfg.catalog}.{cfg.schema}.hotel_booking")
model_udf = mlflow.pyfunc.spark_udf(spark, model_uri)

preds_df = input_df.select(
    col("Booking_ID"),
    model_udf(struct(*[col(c) for c in columns])).alias("Predicted_BookingPrice")
)
```

After we define a `pyspark DataFrame`, we can create a feature table. Here, we use the `databricks-feature-engineering` functionality, but we

could also create a feature table by using SQL commands. In fact, we could use any Delta table with a primary key constraint as a feature table. In our example, we'll use the `hotel_booking_price_preds` feature table to update the online store, and we must enable `ChangeDataFeed` for that purpose.

```
from databricks.feature_engineering import FeatureEngineeringClient

fe = FeatureEngineeringClient()

feature_table_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_price_preds"
fe.create_table(
    name=feature_table_name,
    primary_keys=["Booking_ID"],
    df=preds_df,
    description="Hotel Booking Prices predictions feature table"
)

spark.sql(f"""
    ALTER TABLE {feature_table_name}
    SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
    """)
```

Now that the feature table is available in Unity Catalog, we can create a *FeatureSpec*, which is a user-defined set of features and functions.

Creating a FeatureSpec

To create a `FeatureSpec`, we need a list that consists of instances of `FeatureLookup` and `FeatureFunction`, where the order of elements defines the order of execution. `FeatureSpecs` are stored in and managed by Unity Catalog, and we can view them via the UI in the Databricks Catalog Explorer (just like the tables and models we created earlier).

In our example, we only use `FeatureLookup` to retrieve values based on the key:

```
from databricks.feature_engineering import FeatureLookup

feature_spec_name = f"{cfg.catalog}.{cfg.schema}.return_hotel_booking_prices"
features = [
    FeatureLookup(
```

```

        table_name=feature_table_name,
        lookup_key="Booking_ID",
        feature_names=["Predicted_BookingPrice"],
    )
]
fe.create_feature_spec(name=feature_spec_name,
                      features=features,
                      exclude_columns=None)

```

Note that I will be covering `FeatureFunction` in the next section of this chapter, when we discuss Model Serving with feature lookups.

Feature Serving would not work efficiently without an Online store, which we can initialize now, now that we've stored the `FeatureSpec` in Unity Catalog.

Publishing a Table to a Databricks Online Feature Store

We can create a Databricks Online Feature Store by using the `create_online_store` method from the `FeatureEngineering` client. We can also define the size of the Feature Store. The capacity options correspond to the number of capacity units (each of which is about 2 GB of RAM): "CU_1", "CU_2", "CU_4", or "CU_8".² Notice that we pass the usage policy ID here to ensure that the costs are properly associated with the project:

```

instance_name = "hotel-booking-price-preds"

fe.create_online_store(
    name=instance_name,
    capacity="CU_1",
    usage_policy_id=cfg.usage_policy_id)

```

Since the Databricks Online Feature Store is built on Lakebase, you can also create and manage it by using the Databricks SDK. Make sure to configure the compute used by the branch in your Lakebase project to scale down to zero when idle to avoid unnecessary costs. We'll demonstrate this in [Chapter 7](#), where we use Lakebase to store agent session information.

NOTE

At the time of writing this book, Lakebase is not supported on Databricks Free Edition. You'll therefore need a premium subscription to execute this code.

After we create the online store and verify that its status is `AVAILABLE`, we can synchronize the data from the feature table to the online store by using the `publish_table()` method:

```
online_store = fe.get_online_store(name="hotel-booking-price-preds")
fe.publish_table(
    online_store=online_store,
    source_table_name=feature_table_name,
    online_table_name=f"{feature_table_name}_online"
)
```

Although the `FeatureSpec` definition doesn't explicitly reference an online table, it's implicitly associated with the corresponding feature table.

[Figure 4-3](#) shows the catalog view of the online table, where you can find the reference to the Lakeflow Spark Declarative Pipeline that updates the online table based on the changes in the feature table. Because `ChangeDataFeed` is enabled in the feature table, only incremental updates will be processed.

Catalog Explorer
>
mlops_dev
>
hotel_booking
>

hotel_booking_price_preds_online

☆

⋮

Open in a dashboard

Overview
Columns
Sample Data
Details
Permissions
History
Lineage
Insights
Quality

Description

Database table synced from another UC table

Overview

Synced table status	Online
Source table	mlops_dev.hotel_booking.hotel_booking_price_preds
Primary key(s)	Booking_ID
Sync schedule	Triggered
Instance Name	hotel-booking-price-preds
Postgres database	mlops_dev

Data Ingest

Pipeline id	10081ad7-83e8-4931-ae1d-7b93751e0c57
Update status	Completed
Last processed commit	3
Last processed timestamp	Feb 28, 2026 at 12:19 AM (36 seconds ago) Sync now

Figure 4-3. Catalog view of an online table

Note that the synchronization schedule is set to Triggered and the pipeline gets triggered when you run `publish_table`. If `publish_table` is called with `publish_mode='CONTINUOUS'`, the sync schedule will be Continuous, which means that the online table gets updated once the feature table gets updated.

Triggered pipelines are more cost efficient in most cases because the cluster runs only for the update window but new data isn't processed until the next trigger. Continuous pipelines keep a cluster running at all times, which costs more but delivers lower latency.

It's generally recommended to use triggered synchronization when updates are needed every 10 minutes or less frequently. Choose continuous mode when you need fresh data within 10 seconds to a few minutes.³

Endpoint Deployment

We can create a Feature Serving endpoint in much the same way we created a model-serving endpoint. The main difference lies in what we provide as the `ServedEntityInput`.

For a model, we specify a particular version of a registered model. However, for a Feature Serving endpoint, we use a `FeatureSpec`. A key difference here is that a `FeatureSpec` doesn't have a version number, unlike a registered model. Here again, we provide a serverless usage policy ID:

```
from databricks.sdk.service.serving import (
    EndpointCoreConfigInput,
    ServedEntityInput,
)

served_entities = [
    ServedEntityInput(
        entity_name=feature_spec_name,
        scale_to_zero_enabled=True,
        workload_size="Small",
    )
]

w = WorkspaceClient()
endpoint_name = "hotel-booking-feature-serving"

w.serving_endpoints.create(
    name=endpoint_name,
    config=EndpointCoreConfigInput(
        served_entities=served_entities),
    budget_policy_id=cfg.usage_policy_id
)
```

Finally, we can call the endpoint. You'll notice that the payload in this case is much simpler, while the rest stays the same.

```
import requests

host = w.config.host
token = w.tokens.create(lifetime_seconds=1200).token_value

serving_endpoint = f"{host}/serving-endpoints/{endpoint_name}/invocations"

response = requests.post(
    serving_endpoint,
    headers={"Authorization": f"Bearer {token}"},
    json={"dataframe_split": {"columns": ["Booking_ID"]},
```

```
"data": [{"INN36285"}] }},
```

```
)
```

Now that we have discussed Model Serving and Feature Serving, we can move on to Model Serving with feature lookup, which combines the functionality of the previously discussed architectures.

Model Serving with Feature Lookup

Now, let's explore our most complex scenario, which combines the two approaches we've already covered. In this setup, a model request doesn't need to contain all the features. Instead, some input features are provided with the request itself, while others are automatically retrieved through a *feature lookup* from an online store. The great thing about this is that those features in the online store can be computed in either a batch or streaming fashion, which gives you a lot of flexibility, depending on your needs.

[Figure 4-4](#) shows a simplified architecture for Model Serving with feature lookup. A scheduled Lakeflow job preprocesses data and updates a feature table in Databricks Unity Catalog, which is then synced to the online store. Another scheduled job retrains the model using the latest features and registers it in Unity Catalog. During inference, the Model Serving endpoint fetches features from the online store using a primary key (in our case, `Booking_ID`), combines them with any features provided in the request payload, runs the model's prediction function, and returns the result to the application.

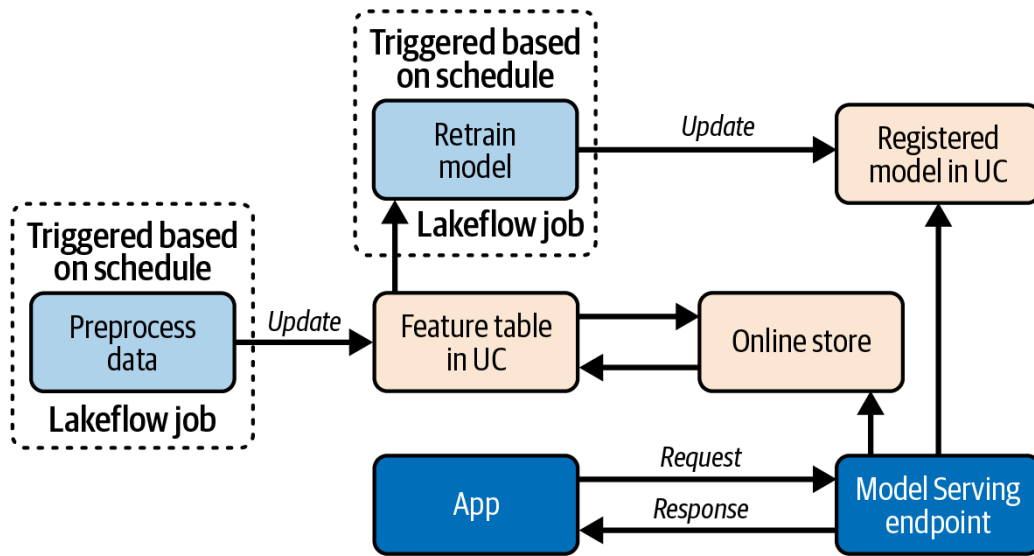


Figure 4-4. Simplified architecture for Model Serving with feature lookup

To implement this architecture, we must log the model using the `databricks-feature-engineering` package. This requires us to define a training set object, where we specify a sequence of feature lookups and feature functions.

I won't cover the model endpoint deployment here, as it is nearly identical to the standard Model Serving scenario. The key difference appears at inference time: you don't need to pass all features in the request. Instead, you must provide a `Booking_ID` as an additional input. This is because some features are either computed dynamically or retrieved from the online feature table, and `Booking_ID` is required to perform those lookups.

Let's walk through the `chapter4_3.model_serving_feature_lookup.py` notebook, where we'll implement all these steps.

We'll start by loading the data. Notice that we drop several features from the training set—specifically, those that will later be computed or looked up: `lead_time`, `arrival_month`, and `repeated`. Later in the notebook, we define feature functions to calculate `lead_time` and `arrival_month` based on the reservation and arrival dates, and we retrieve the `repeated` value from the feature table.

```

from datetime import datetime

from pyspark.sql import SparkSession

from hotel_booking.config import ProjectConfig

```

```

from hotel_booking.data.data_loader import DataLoader
from hotel_booking.utils.common import set_mlflow_tracking_uri

set_mlflow_tracking_uri()
cfg = ProjectConfig.from_yaml(config_path="../project_config.yml")

spark = SparkSession.builder.getOrCreate()

data_loader = DataLoader(spark=spark, config=cfg)
train_query, test_query = data_loader.generate_queries()
train_set = spark.sql(train_query).drop(
    "lead_time", "arrival_month", # computed
    "repeated", # looked up
    # present in the original table but not used:
    "P_C", "P_not_C", "booking_status")
test_set = spark.sql(test_query).toPandas()

```

Notice that we also drop features that are present in the DataFrame returned by the `train_query` execution. While this may seem counterintuitive, it prevents those columns from becoming required inputs at serving time. When you create a training set with the model and later log the model using the feature-engineering client, Databricks uses the training set schema (excluding the label) as the model's input contract. As a result, every column present during training is expected in inference requests, even if the model does not actually use it.

Let's start with defining feature functions.

Creating Feature Functions and a Feature Table

A *feature function* is a construct we use to create a training set with the feature-engineering client. It allows us to define how certain features should be computed dynamically as part of the feature pipeline.

A feature function relies on a user-defined function (UDF) that's registered in Databricks Unity Catalog. This ensures that the function will be governed, reusable, and accessible during both training and inference.

Let's start by defining UDFs for calculating the `lead_time` and `arrival_month` features:

```

lead_time_function = f"{cfg.catalog}.{cfg.schema}.calculate_lead_time"

```

```

spark.sql(f"""
CREATE OR REPLACE FUNCTION {lead_time_function}
(arrival_date TIMESTAMP, reservation_date TIMESTAMP)
RETURNS BIGINT
LANGUAGE PYTHON AS
$$
return (arrival_date-reservation_date).days
$$""")

arrival_month_function = f"{cfg.catalog}.{cfg.schema}.get_arrival_month"

spark.sql(f"""
CREATE OR REPLACE FUNCTION {arrival_month_function}
(arrival_date TIMESTAMP)
RETURNS BIGINT
LANGUAGE PYTHON AS
$$
return arrival_date.month
$$""")

```

We can define feature functions by using either SQL or Python. However, for feature-engineering workflows, only Python-based feature functions are supported.

Next, let's create a feature table that contains historical booking features. We'll assume that returning customers may receive a discount on new bookings, which can affect the price per night. Previously, we created a feature table by using the feature engineering client. This time, we'll create the feature table directly by using SQL, with the `Booking_ID` defined as the primary key.

But before assigning `Booking_ID` as the primary key, we must first apply a `NOT NULL` constraint. In Databricks, the `NOT NULL` constraint is *enforced*, meaning that any attempt to insert or update a row with a null value in that column will fail. In contrast, the primary key constraint is *informational*, which means that Databricks records the metadata and can use it for query optimization but does not validate or enforce uniqueness. As a result, we can still insert duplicate primary key values unless uniqueness is enforced through other mechanisms in our data pipeline.⁴

```

feature_table_name = f"{cfg.catalog}.{cfg.schema}.historical_booking_features"
spark.sql(f"""
    CREATE OR REPLACE TABLE {feature_table_name}

```

```

        AS SELECT Booking_ID, repeated FROM
        {cfg.catalog}.{cfg.schema}.hotel_booking
    """)
    spark.sql(f"""
        ALTER TABLE {feature_table_name}
        ALTER COLUMN Booking_ID SET NOT NULL
    """)
    spark.sql(f"""
        ALTER TABLE {feature_table_name}
        ADD CONSTRAINT pk_Booking_ID PRIMARY KEY (Booking_ID)
    """)

```

We must also enable the `ChangeDataFeed` to synchronize the table with the online store, which is required for the Model Serving with feature lookup:

```

    spark.sql(f"""
        ALTER TABLE {feature_table_name}
        SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
    """)

```

With our features and UDFs in place, the next step is to define a training set. This is an important step for reproducibility, as it captures the input PySpark DataFrame along with the features, transformations, and labels used. By doing this, we ensure lineage and consistency between our training and inference processes.

Defining the Training Set

We use the `create_training_set()` function to define a training set. We pass a list that consists of instances of `FeatureLookup` and `FeatureFunction`, in which the order of the list determines the order of execution. `FeatureFunction` takes the `udf_name` as an argument, and it also requires the *output name* (what the feature is called in the training set) and the input bindings.

The keys in the `input_bindings` dictionary refer to the names of inputs specified in the UDF, while the values refer to the feature names in the input DataFrame:

```

from databricks.feature_engineering import (
    FeatureEngineeringClient,
    FeatureFunction,
    FeatureLookup,
)

fe = FeatureEngineeringClient()

training_set = fe.create_training_set(
    df=train_set,
    label=cfg.target,
    feature_lookups=[
        FeatureLookup(
            table_name=feature_table_name,
            feature_names=["repeated"],
            lookup_key="Booking_ID",
        ),
        FeatureFunction(
            udf_name=lead_time_function,
            output_name="lead_time",
            input_bindings={"arrival_date": "arrival_date",
                           "reservation_date": "date_of_reservation"}
        ),
        FeatureFunction(
            udf_name=arrival_month_function,
            output_name="arrival_month",
            input_bindings={"arrival_date": "arrival_date"},
        ),
    ],
)

```

Since we are training a LightGBM model, we use the `load_df()` method of the training set to obtain a PySpark DataFrame, and then, we call `toPandas()` to convert it into a Pandas DataFrame. Notice that we include an additional feature, `repeated`, when defining `X_train` and `X_test`. We have not used this feature previously, so we introduce it here to showcase how we can retrieve it via feature lookup from the feature table:

```

training_df = training_set.load_df().toPandas()
X_train = training_df[cfg.num_features + cfg.cat_features + ["repeated"]]
y_train = training_df[cfg.target]
X_test = test_set[cfg.num_features + cfg.cat_features + ["repeated"]]
y_test = test_set[cfg.target]

```

Once we have loaded the data, we can train the model as usual:

```
from hotel_booking.models.lightgbm_model import LightGBMModel

model = LightGBMModel(config=cfg)
model.train(X_train=X_train, y_train=y_train)
```

The only difference is in the model logging step. Previously, we used standard MLflow functions to log the model. Now, we log the model using the `FeatureEngineeringClient` and pass the training set as an additional input.

In this example, we provide dummy values for `branch` and `git_sha`:

```
import mlflow
import numpy as np
from mlflow.models import infer_signature
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from hotel_booking.config import Tags

mlflow.set_experiment("/Shared/hotel-booking-training-fe")
tags = Tags(**{"git_sha": "1234567890abcd", "branch": "main"})

run = mlflow.start_run(
    run_name=f"lightgbm-training-{datetime.now().strftime('%Y-%m-%d')}",
    description="LightGBM model training",
    tags=tags.to_dict(),
)
run_id = run.info.run_id
mlflow.log_params(model.parameters)
signature = infer_signature(
    model_input=X_train,
    model_output=model.pipeline.predict(X_train[0:1])
)
y_pred = model.pipeline.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
metrics = {
    "mse": mse,
    "rmse": rmse,
    "mae": mean_absolute_error(y_test, y_pred),
    "r2_score": r2_score(y_test, y_pred),
}
```

```
mlflow.log_metrics(metrics)

fe.log_model(
    model=model.pipeline,
    flavor=mlflow.sklearn,
    artifact_path="lightgbm-pipeline-model-fe",
    training_set=training_set,
    signature=signature,
)
```

In a production deployment, however, these should be real values that are automatically passed from the GitHub Actions workflow via the DAB. We will discuss this setup in the next chapter.

It's interesting to examine how the model is logged (see [Figure 4-5](#)). The model is stored with the `python_function` flavor, and it uses the `databricks.feature_store.mlflow_model` loader.

In addition to the standard *MLmodel* file and environment definition, the logged artifacts include a *metadata/* folder (which duplicates some of the top-level information) and a *data/* folder. The *data/* folder contains the serialized raw model as well as a *feature_spec.yaml* file that defines the feature lookups and feature functions required at inference time.

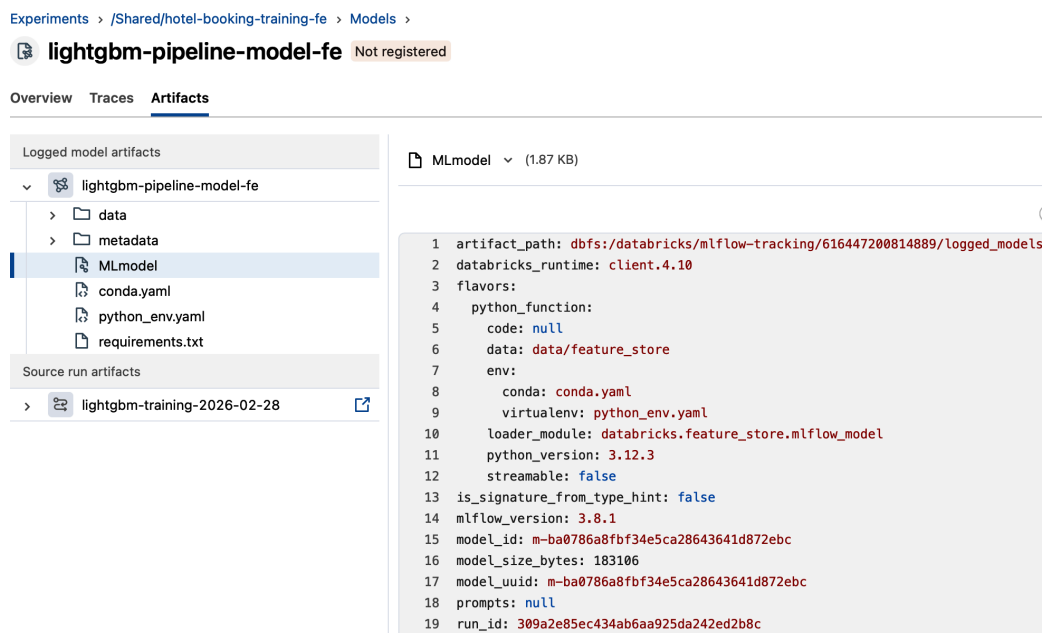


Figure 4-5. UI view of the model logged with feature-engineering package

The *feature_spec.yaml* file captures each input feature's data type, topological ordering, and source. For example, the `number_of_adults` feature exists in the training data, so its source is `training_data`:

```
- number_of_adults:
    data_type: bigint
    topological_ordering: 5
    source: training_data
```

The `repeated` feature is looked up from a feature table, so the source is `feature_store`, with the table and lookup key specified:

```
- repeated:
    table_name: mlops_dev.hotel_booking.historical_booking_features
    feature_name: repeated
    lookup_key:
        - Booking_ID
    data_type: bigint
    topological_ordering: 14
    source: feature_store
```

The `lead_time` feature is produced by an on-demand feature function, so the source is `on_demand_feature`, with the UDF and its input bindings defined:

```
- lead_time:
    udf_name: mlops_dev.hotel_booking.calculate_lead_time
    input_bindings:
        arrival_date: arrival_date
        reservation_date: date_of_reservation
    data_type: bigint
    topological_ordering: 13
    source: on_demand_feature
```

Registering the Model and Publishing an Online Table

After we log the model, we register it so that we can use the model version behind a Model Serving endpoint:

```
from mlflow import MlflowClient

model_name = f"{cfg.catalog}.{cfg.schema}.hotel_booking_model_fe"
registered_model = mlflow.register_model(
    model_uri=f"runs://{run_id}/lightgbm-pipeline-model-fe",
```

```

        name=model_name,
        tags=tags.to_dict(),
    )

    client = MlflowClient()
    client.set_registered_model_alias(
        name=model_name,
        alias="latest-model",
        version=registered_model.version,
    )

```

Lastly, we must publish an online table to make it reachable for the feature lookup at the serving stage. In this example, we reuse the same online store we created earlier:⁵

```

online_store = fe.get_online_store(name="hotel-booking-price-preds")

fe.publish_table(
    online_store=online_store,
    source_table_name=feature_table_name,
    online_table_name=f"{feature_table_name}_online")

```

When it comes to Model Serving, the deployment process remains the same as what we covered earlier in the Model Serving section. The only difference lies in the input payload, so let's explore it in detail.

Payload Structure

For Model Serving with feature lookup, we do not need to provide values for `lead_time`, `arrival_month`, or `repeated`, as these features are computed or retrieved automatically at inference time. Instead, we supply the raw inputs required for those transformations: specifically, `arrival_date` and `date_of_reservation` for the feature functions and `Booking_ID` for the feature lookup. [Table 4-3](#) highlights the differences between the input and output payloads across the three serving scenarios we've covered.

Table 4-3. Payloads for Model Serving, Feature Serving, and Model Serving with feature lookup

Scenario	Input payload	Output payload
Model Serving (PyFunc)	<pre>{ "dataframe_split": { 'columns': ['number_of_adults', 'number_of_ children', 'number_of_weekend_nig hts', 'number_of_week_nights', 'c ar_parking_space', 'special_reque sts', 'lead_time', 'type_of_mea l', 'room_type', 'arrival_month', 'market_segment_type'], 'data': [[2, 0, 0, 1, 0, 0, 103, 'Not Sel ected', 'Room_Type 1', 8, 'Onlin e']] } }</pre>	<pre>{ "predictions": { "Total price per nig ht": [104.98] } }</pre>
Feature Serving	<pre>{ "dataframe_split": { 'columns': ['Booking_ID'], 'data': [['INN3 6285']] } }</pre>	<pre>{ "outputs": [{"Booking_I D": "INN3628 5", "Predicted_BookingPric e": 104.98 }] }</pre>
Model Serving with feature lookup	<pre>{ "dataframe_split": { 'columns': ['Booking_ID', 'number_of_adult s', 'number_of_children', 'number _of_weekend_nights', 'number_of_w eek_nights', 'car_parking_space', 'special_requests', 'type_of_mea l', 'room_type', 'market_segment_ type', 'arrival_date', 'date_of_r eservation'], 'data': [['INN362 85', 2, 0, 0, 1, 0, 0, 'Not Selec ted', 'Room_Type 1', 'Online', '2 018-08-01 00:00:00', '2018-03-01 00:00:00']] } }</pre>	<pre>{ "predictions": { [104.9 8] } }</pre>

It's important to note that both the Feature Serving endpoint and the Model Serving endpoint with feature lookup will fail to return predictions if the `Booking_ID` is missing from the feature table. To handle this, you can implement a `FeatureFunction` to substitute default values for missing entries. Alternatively, you can create a custom Python function

(PyFunc) model that interacts directly with Lakebase, which is the underlying system used for feature lookups.

So far, we’ve been using a personal access token to call the endpoint. This is great for testing, but it’s not recommended for production environments. Furthermore, route-optimized endpoints can only be queried with an OAuth token. Let’s walk through how to set this up.

OAuth Authentication

Personal access tokens are powerful because they grant the holder all the permissions of the associated user. A more secure approach is to use an OAuth token associated with the service principal, which has no permissions by default. You can then grant it permissions to query a specific endpoint, just as you would a user.

Workspace admins have the required permissions to create a Databricks service principal.⁶ Once they create the service principal, they must update the endpoint permissions. They can accomplish both actions via Databricks API, Databricks SDK for Python, or the Databricks Terraform module, or through the workspace settings and the endpoint’s permission settings in the UI. [Figure 4-6](#) depicts the “Service principals” view in the UI.

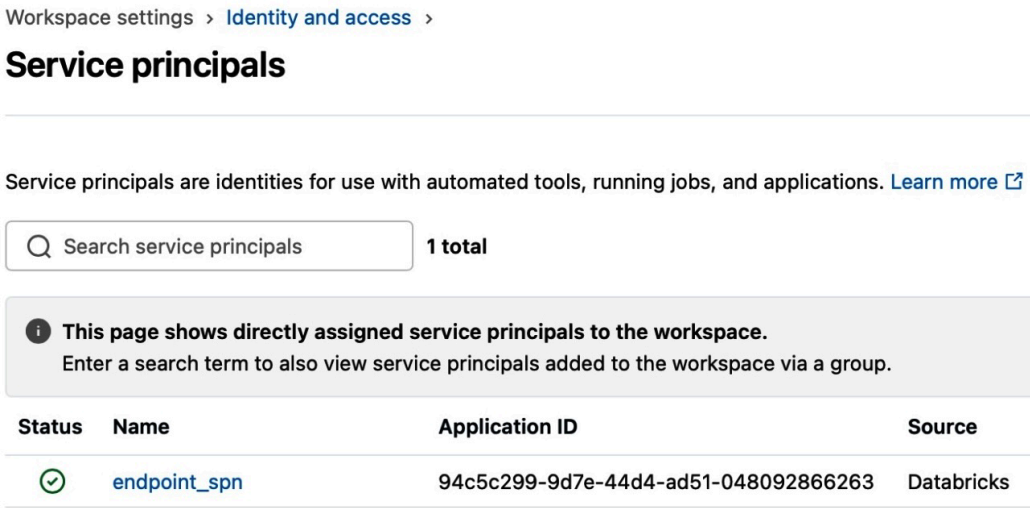


Figure 4-6. “Service principals” view in the UI

[Figure 4-7](#) depicts the Permission Settings view in the UI for an individual endpoint (`hotel-booking-pyfunc`).

Permission Settings

NAME	PERMISSION	
Maria Vechtomova (maria@cauchy.io)	Can Manage	
Admins	Can Manage	inherited
endpoint_spn (94c5c299-9d7e-44d4-ad51-048092866263)	Can Query	
Select user, group or service principal...	Can Query	Add
	Cancel	Save

Figure 4-7. Object’s “Permission Settings” view in the UI

Once we’ve created the service principal and set the permissions, we need to create a client secret, which is required for authentication. We can do this [programmatically](#) or via the “Service principal details” page in the UI (see [Figure 4-8](#)). The secret can have a maximum lifetime of 730 days. Also remember that one of the most common system failures is due to expired secrets, and there must be a process in place to refresh the secrets periodically before they expire.

Workspace settings > [Identity and access](#) > [Service principals](#) >

Service principal details

endpoint_spn

Delete

Configurations

Permissions

Secrets

Git integration

OAuth secrets

OAuth secrets can be used to secure authentication to the Databricks API. [Learn more](#)

ID	Created at	Expires at
----	------------	------------

Figure 4-8. “Service principal details: Secrets” view in the UI

Let’s store the values of `DATABRICKS_CLIENT_ID` (the same as in `Application ID`) and `DATABRICKS_CLIENT_SECRET` in the `.env` file, so that we can run the following code from the VS Code to get an access token:⁷

```
import os

import requests
```

```

from dotenv import load_dotenv
from requests.auth import HTTPBasicAuth

load_dotenv()

def get_token():
    response = requests.post(
        f"https://{os.environ['DATABRICKS_HOST']}/oidc/v1/token",
        auth=HTTPBasicAuth(
            os.environ["DATABRICKS_CLIENT_ID"],
            os.environ["DATABRICKS_CLIENT_SECRET"]
        ),
        data={
            'grant_type': 'client_credentials',
            'scope': 'all-apis'
        }
    )
    return response.json()["access_token"]

token = get_token()

```

The token we fetched will be valid for one hour. For more details on OAuth authentication, you can refer to the [Databricks OAuth documentation](#).

Now, before we move on, let's discuss some important practicalities. You need to be aware of Databricks' serving limitations when designing your applications, so let's cover those next.

Serving Limitations

Databricks Model Serving offers a scalable way to deploy ML models, but it comes with constraints that are worth understanding up front, particularly when migrating from existing infrastructure.

A few are likely to surface during a migration:

Request format

Endpoints expect a predefined request payload schema. As we saw earlier, this can require adaptation if your application already uses a custom or incompatible format.

Networking

Private connectivity is not always available out of the box and may need additional configuration, depending on your cloud and workspace setup.

CPU memory

By default, each CPU model endpoint is allocated 4 GB of memory per worker. The `CPU_MEDIUM` and `CPU_LARGE` workload types let you trade concurrency for more memory on the same CPU hardware, which matters when a model does not fit within the standard allocation—a common consideration in multimodel deployments.

Payload and logging size

Requests are limited to 16 MB each. Separately, requests or responses larger than 1 MB are not captured in inference tables, so very large payloads can serve successfully but fall outside your logged data.

These are mostly design time decisions rather than blockers, but they are easier to account for early than to retrofit later. The [Databricks documentation](#) lists the full set of current limits.

None of this should suggest that the platform is constrained at the top end. Concurrency is managed at the workspace level, with a default provisioned concurrency of 4,096 units per workspace and 1,024 per model, and a single endpoint can reach up to 300,000 queries per second (QPS) with route optimization. Most of these limits can be raised on request.

GPU serving, though outside the scope of this book, has matured along the same lines, and it now backs some genuinely demanding production workloads. Superhuman is a good example.⁸ It migrated its grammar correction model from a self-managed vLLM stack to Databricks Model Serving, and now, it serves it at more than 200,000 QPS, with subsecond P99 latency and four nines of availability.

Conclusion

In this chapter, we explored the various Model Serving architectures that are available on Databricks, starting with basic approaches such as Model

Serving and Feature Serving and progressing to more advanced patterns like Model Serving with feature lookup. We also discussed key topics including payload structure, authentication, and serving limitations.

Let's now summarize when each serving approach is appropriate:

Feature Serving

This is useful when predictions are precomputed and simply need to be retrieved at request time, or when a lightweight transformation is required that doesn't involve a model.

Model Serving

This is the right choice when predictions need to be computed in real time and all required features are provided directly in the request payload.

Model Serving with feature lookup

This is appropriate when predictions are computed on demand but some features must be fetched from an online feature store during inference.

The different options fit different operational requirements depending on latency, infrastructure, and the availability of features at the time of prediction.

Now, with Model Serving in place, our next step is to bring everything together into a robust, automated deployment workflow.

¹ For a detailed discussion, see Alex Deng, *Causal Inference and Its Applications in Online Industry* (self-pub online, 2023), ch. 8, “Statistical Analysis of A/B Tests,” <https://oreil.ly/XlMkP>.

² The Lakebase Autoscaling model is the default for the Databricks Online Feature Store. For information about autoscaling, see [the Databricks documentation](#).

³ This is a generic recommendation for Lakeflow Spark Declarative Pipelines, per [the Databricks documentation](#).

⁴ Consult [the Databricks documentation on constraints](#) for more details.

- 5 For cost efficiency reasons during development, you should make sure to enable scale to 0 on Lakebase compute. See [the Databricks documentation](#) for details.
- 6 For Databricks on Azure, it is common to use Microsoft Entra ID service principals instead and sync them automatically to the Databricks account using automatic identity management. Consult [the Databricks documentation](#) for more details.
- 7 Make sure to add the `.env` file to `.gitignore` to avoid accidentally committing secrets to your source code repository. While `.env` files are convenient for local development, production environments should use a dedicated secret management solution, such as a Key Vault, to securely store and manage credentials, API keys, and other sensitive information.
- 8 Read the blog [“How Superhuman and Databricks built a 200K QPS inference platform together”](#).

Previous chapter

Next chapter

< [3. MLflow for Traditional ML](#)

[5. Machine Learning Model Deployment](#) >